

Design and Development of a 6DOF Wi-Fi Controlled Robotic Arm with Camera Feedback

Aidan Hernandez

Professor Parvaneh Ghaforyfard

ENG GEN 101

29 May 2026

Introduction

The purpose of this project was to design and build a robotic arm that could be controlled remotely through Wi-Fi via a GUI and stream live camera feedback. The project was a combination of mechanical design, mechatronics, and software development into a single working prototype. The final system was designed using the engineering design process and is intended to solve the problem of taking off a finished 3D printed object from the print bed so I can remotely print another object while being away from home. However, I have applied a real-world use case of technology to solve the problem of inspecting dangerous equipment by making this process remote and automated.

Engineering Design Process

The engineering design process began by identifying the problem. In many real-world situations, a person may need to move an object, inspect a space, or interact with equipment without standing directly next to it due to a variety of reasons. This can be useful in manufacturing, education, hazardous industrial environments, and remote operations. A robotic arm is a practical solution because it can reproduce controlled motion or be operated live while keeping the operator safe. My background research focused on robotic arm structures, servo and continuous motor control, Wi-Fi communication, and camera-based remote operations. Existing systems were reviewed to understand how robotic arms are powered, how motion is controlled, and how feedback is provided to the user. This included looking into robotic arms controlled by stepper motors and DIY projects done by others such as the PAROL 6; an open-source robotic arm that can be ordered as a kit. Research also included the use of ESP32 microcontrollers, which integrate Wi-Fi and Bluetooth capability in a compact embedded platform (Espressif), and the PCA9685 servo driver board, which can control up to 16 servo channels over I2C with only

two signal pins (Adafruit, “Overview”). After researching the problem, several solutions were considered. A wired robotic arm was one option; however, it would have limited remote capabilities when compared to a Wi-fi or Bluetooth controlled arm. A Bluetooth solution was also considered, but Wi-Fi was selected because it allowed the system to run through a locally hosted laptop server and made it easier to separate the camera's feed from the control commands.

The final design used a Python-based GUI (graphical user interface), an ESP32- CAM for video streaming, and a separate ESP32 for receiving arm commands. The robotic arm was designed on Onshape as a 6 degree of freedom system and programming for the Flask server locally hosted on a laptop through PyCharm. The structure was modeled to match the arm’s physical movement requirements and to keep the assembly compact and printable. The final design used seven motors total: three REV Robotics continuous motors, two MG996R servo motors, and two micro servo motors. The motors were assigned to the base, shoulder, elbow, wrist, and claw functions. This arrangement was chosen in accordance with the inverse kinematics from a robotic arm simulator on the GeoGebra website made by Oscar Arcera López.

The prototype was built using the 3D-printed arm parts, the ESP32-CAM with antenna, a separate ESP32 development board, a PCA9685 servo driver, and a 5V 20A external power supply. The power supply was necessary because the motors and electronics required more current than a microcontroller USB port could safely provide. The ESP32 board handled the command side of the system, while the ESP32-CAM handled the video side. The Python GUI, built in PyCharm, displayed the camera feed on the left and arm controls on the right, allowing the user to monitor and operate the arm from one screen.

Testing and evaluation were completed by checking Wi-Fi connection stability, live video streaming, motor response, and full arm motion. The arm successfully responded to control

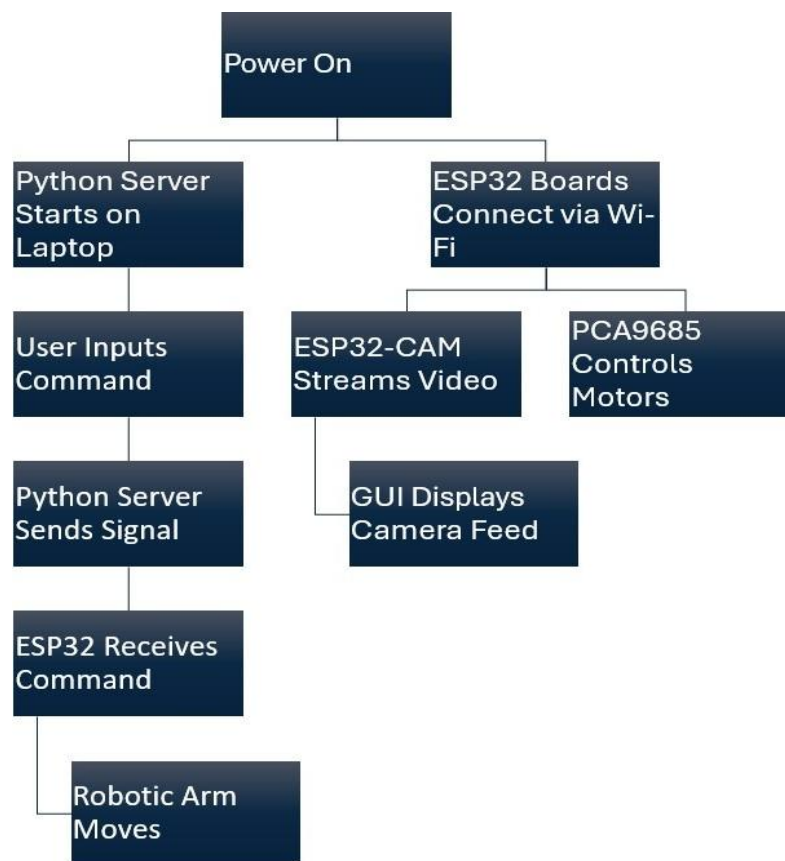
inputs, and the camera feed allowed the operator to see the movement in real time. During testing, some of the main weaknesses involved power distribution, synchronization between multiple motors, and mechanical alignment of joints. These issues were addressed through adjustment, calibration, and redesign of the mounting positions. Future improvements could include stronger motors, better cable management, battery-powered portability, and additional sensors for autonomous movement. A mobile app version of the control system would also make the project easier to use and more flexible in a demonstration setting.

Technical Discussion

The system works by separating visual feedback from control communication. The ESP32-CAM streams video to the locally hosted Python server, and the second ESP32 receives movement commands from the same server. The Python program serves as the central interface between the operator and the hardware. When the user presses a control on the GUI, the program sends a command to the ESP32. The ESP32 then forwards the motion request to the PCA9685 driver, which generates the PWM signals needed to control the motors.

PWM, or pulse width modulation, is important because it allows the controller to send position or motion instructions using timed electrical pulses. This is the standard method used by many servo systems. The PCA9685 driver is especially useful because it offloads the repetitive signal generation from the microcontroller and can manage many output channels at once. In this project, that made it easier to support multiple motors without overloading the controller. The use of a separate camera board also reduced complexity because the video stream could be handled independently from the arm motion commands.

The project has several real-world applications. A system like this could be adapted for basic laboratory automation, hazardous handling, object inspection, or educational robotics demonstrations. It also reflects the type of system integration used in larger robotic platforms, where mechanical motion, embedded control, and operator feedback must all work together. The main advantage of the design is that it is low cost, modular, and easy to expand. The main limitations are that it depends on a stable Wi-Fi connection, requires a strong external power supply, and is limited by the torque and precision of the motors used.



Results and Discussion

The final project accomplished the main goal of creating a remotely controlled robotic arm with live camera feedback. The arm was designed, modeled, assembled, and operated as a

functional prototype. The Python GUI provided a practical way to control the arm, and the ESP32-CAM provided the operator with a live view of the workspace. The result was a system that demonstrated the engineering design process from problem definition through testing and redesign.

Several challenges were encountered during development. One challenge was ensuring that the motors received enough current without causing the microcontrollers to reset. Another challenge was mechanical alignment since the arm parts had to be positioned carefully for the joints to move correctly. The most important lesson learned was that hardware and software must be developed together. Even if the code is correct, the system will not work well if the mechanical design is weak, or the power supply is undersized. Likewise, a strong mechanical design still requires reliable software and communication to function.

Overall, the project was successful because it met the main design objectives and demonstrated multiple engineering concepts in one system. It showed how CAD design, embedded systems, wireless communication, and prototype testing can be combined into an engineering solution. The project also showed that iteration is an important part of engineering since improvements were made after testing revealed specific weaknesses.

Conclusion

This project successfully developed a 6DOF Wi-Fi controlled robotic arm with camera feedback using Onshape modeling, ESP32-based communication, a PCA9685 servo driver, a Python GUI, and an external high-current power supply. The arm was built as an integrated

system that could receive wireless commands, stream live video, and move multiple joints in a coordinated way. The final design demonstrated both the technical and practical value of the engineering design process. The importance of the project was not only in building a robotic arm, but also in showing how engineering problems are solved through research, testing, and redesign. The process made it clear that successful engineering depends on balancing performance, cost, power, and usability. In the end, the project provided a strong example of how science, engineering, and technology can be combined to create a working solution to a real problem.

Works Cited

Adafruit. "Overview." *Adafruit PCA9685 16-Channel Servo Driver*, Adafruit Learning System, 16 Oct. 2012, learn.adafruit.com/16-channel-pwm-servo-driver/overview. Accessed 29 May 2026. [[Design and...edback \(1\) | PDF](#)]

Arcera López, Oscar. "Robotic Arm Simulation." *GeoGebra*, www.geogebra.org/m/fsq5cjag. Accessed 29 May 2026. [learn.adafruit.com]

Espressif Systems. "ESP32 Wi-Fi & Bluetooth SoC." *Espressif Systems*, www.espressif.com/en/products/socs/esp32. Accessed 29 May 2026. [geogebra.org]

JetBrains. "PyCharm: The Only Python IDE You Need." *JetBrains*, www.jetbrains.com/pycharm/. Accessed 29 May 2026. [espressif.com]

MLA Handbook. 9th ed., Modern Language Association of America, 2021.

Modern Language Association. "MLA Style Center." *Modern Language Association*, style.mla.org/. Accessed 29 May 2026.

Onshape. "Onshape Product Development Platform." *Onshape*, www.onshape.com/en/. Accessed 29 May 2026. [jetbrains.com]

Pallet Projects. "Welcome to Flask." *Flask Documentation*, flask.palletsprojects.com/. Accessed 29 May 2026. [onshape.com]

Python Software Foundation. "Python Documentation." *Python*, www.python.org/doc/. Accessed 29 May 2026.

REV Robotics. "HD Hex Motor 40:1 Spur Gearbox." *REV Robotics*, www.revrobotics.com/rev-41-1301/. Accessed 29 May 2026. [[flask.pall...objects.com](https://flask.palletsprojects.com)]

Source Robotics. "PAROL6 Docs." *PAROL6 Documentation*, source-robotics.github.io/PAROL-docs/. Accessed 29 May 2026. [revrobotics.com]

Tower Pro. "MG996R." *Tower Pro*, towerpro.com.tw/product/mg996R/. Accessed 29 May 2026. [[source-rob....github.io](https://source-rob...github.io)]